Lab Assignmnet-11.1

Name: R.Vineesha

Hall.No:2303A51873

Batch:14

Task Description #1 – Stack Implementation

Task: Use AI to generate a Stack class with push, pop, peek, and is_empty methods.

Sample Input Code:

```
class Stack:
pass
```

Expected Output:

• A functional stack implementation with all required methods and docstrings.

Prompt

Generate a Python class named Stack that implements a stack data structure using a list.

Include the following methods:

• push(item) – to add an element

• pop() – to remove and return the top element

• peek() – to return the top element without removing it

• is_empty() – to check if the stack is empty

Also include proper docstrings and example usage with input and output.

#Generate a Python class named Stack tha.py

#Generate a Python class named Stack tha.py > ...

```python


class Stack:
    """
    Stack implementation using Python list.
    Follows LIFO (Last In First Out) principle.
    """

    def __init__(self):
        """Initialize empty stack"""
        self.items = []

    def push(self, item):
        """Add element to stack"""
        self.items.append(item)
        print(item, "pushed to stack")

    def pop(self):
        """Remove and return top element"""
        if self.is_empty():
            print("Stack is empty")
            return None
        return self.items.pop()

    def peek(self):
        """Return top element without removing"""
        if self.is_empty():
            print("Stack is empty")
            return None
        return self.items[-1]

    def is_empty(self):
        """Check if stack is empty"""
        return len(self.items) == 0


# Input and Test
s = Stack()

s.push(10)
s.push(20)
s.push(30)

print("Top element is:", s.peek())

print("Popped element is:", s.pop())

print("Stack empty:", s.is_empty())
```

Output:

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
PS D:\AI coding\Lab-11> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/Lab-11/#Genera
10 pushed to stack
20 pushed to stack
30 pushed to stack
Top element is: 30
Popped element is: 30
Stack empty: False
PS D:\AI coding\Lab-11>
```

Observation

• The Stack data structure was successfully implemented using a Python list.

• The **push() method** correctly added elements (10, 20, 30) to the stack.

• The **peek() method** correctly displayed the top element (30) without removing it.

• The **pop() method** successfully removed the last inserted element (30), confirming the LIFO (Last In First Out) principle.

• The **is_empty() method** correctly returned False, indicating the stack still contained elements.

• The program executed without errors and produced the expected output.

• AI-assisted implementation helped in generating structured, readable, and well-documented code efficiently.

Task Description #2 – Queue Implementation

Task: Use AI to implement a Queue using Python lists.

Sample Input Code:

```
class Queue:
pass
```

Expected Output:

• FIFO-based queue class with enqueue, dequeue, peek, and size methods.

Prompt

Generate a Python class named Queue that implements a queue data structure using a list.

Include the following methods:

• enqueue(item) – to add an element to the queue

• dequeue() – to remove and return the front element

• peek() – to return the front element without removing it

• size() – to return the number of elements in the queue

Also include proper docstrings and example usage with input and output.

```python

class Queue:
    """
    Queue implementation using Python list.
    Follows FIFO (First In First Out) principle.
    """

    def __init__(self):
        """Initialize empty queue"""
        self.items = []

    def enqueue(self, item):
        """Add element to rear of queue"""
        self.items.append(item)
        print(item, "enqueued")

    def dequeue(self):
        """Remove and return front element"""
        if self.is_empty():
            print("Queue is empty")
            return None
        return self.items.pop(0)

    def peek(self):
        """Return front element without removing"""
        if self.is_empty():
            print("Queue is empty")
            return None
        return self.items[0]

    def size(self):
        """Return number of elements"""
        return len(self.items)

    def is_empty(self):
        """Check if queue is empty"""
        return len(self.items) == 0


# Input and Test
q = Queue()

q.enqueue(10)
q.enqueue(20)
q.enqueue(30)

print("Front element is:", q.peek())

print("Dequeued element is:", q.dequeue())

print("Queue size is:", q.size())
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE                    Python

Queue size is: 2
PS D:\AI coding\Lab-11>


                    > & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/Lab-11/#Generate a Python class named Stack tha.py"
10 enqueued
20 enqueued
30 enqueued
Front element is: 10
Dequeued element is: 10
Queue size is: 2
PS D:\AI coding\Lab-11>
```

Observation

The queue was successfully implemented using a Python list. The enqueue operation added elements correctly, and the dequeue operation removed the first inserted element, confirming the FIFO principle. The peek method displayed the front element, and the size method returned the correct number of elements. The program executed successfully and produced the expected output.

Task Description #3 – Linked List

Task: Use AI to generate a Singly Linked List with insert and display methods.

Sample Input Code:

class Node:

pass

class LinkedList:

pass

Expected Output:

- A working linked list implementation with clear method documentation.

Prompt

#Generate a Python class named Stack tha.py

D:\AI coding\Lab-11\#Generate a Python class named Stack tha.py

```python
#Use AI to generate a Singly Linked List in Python by creating a Node class and a LinkedList class. Implement methods to insert
# Node class represents each element in the linked list
class Node:
    def __init__(self, data):
        """
        Constructor to initialize node with data and next pointer
        data : value stored in node
        next : pointer to next node (initially None)
        """
        self.data = data
        self.next = None


# LinkedList class to manage nodes
class LinkedList:
    def __init__(self):
        """
        Constructor to initialize empty linked list
        head : starting node of linked list
        """
        self.head = None

    def insert(self, data):
        """
        Insert a new node at the end of the linked list

        Steps:
        1. Create a new node
        2. If list is empty, make it head
        3. Else traverse to last node
        4. Attach new node at end
        """
        new_node = Node(data)

        # If list is empty
        if self.head is None:
            self.head = new_node
        else:
            temp = self.head
            while temp.next:
```

```python
            while temp.next:
                temp = temp.next
            temp.next = new_node

    def display(self):
        """
        Display all nodes in linked list

        Steps:
        1. Start from head
        2. Traverse until None
        3. Print each node data
        """
        temp = self.head

        if temp is None:
            print("Linked List is empty")
            return

        while temp:
            print(temp.data, end=" -> ")
            temp = temp.next

        print("None")


# Main Program
ll = LinkedList()

# Inserting elements
ll.insert(10)
ll.insert(20)
ll.insert(30)
ll.insert(40)

# Display list
print("Linked List Elements:")
ll.display()
```

Output:

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

class named Stack tha.py"
Linked List Elements:
10 -> 20 -> 30 -> 40 -> None
PS D:\AI coding\Lab-11>
```

Observation

- The Singly Linked List was successfully implemented using Node and LinkedList classes.
- The insert() method correctly adds elements at the end of the list.
- The display() method correctly traverses and prints all elements.
- The linked list maintains proper connections between nodes using pointers.
- The output confirms FIFO order of insertion.
- Linked list allows dynamic memory allocation unlike arrays.

Task Description #4 – Binary Search Tree (BST)

Task: Use AI to create a BST with insert and in-order traversal methods.
Sample Input Code:
class BST:

pass
Expected Output:
• BST implementation with recursive insert and traversal methods.

Prompt

Use AI to create a Binary Search Tree (BST) in Python with recursive insert and in-order traversal methods to store and display elements in sorted order.

#Use AI to create a Binary Search Tree (.py

```python
#Use AI to create a Binary Search Tree (BST) in Python with recursive insert and in-order traversal methods to store and display elements in sorted order.
# Node class for BST
class Node:
    def __init__(self, data):
        """
        Initialize node with data and left, right pointers
        """
        self.data = data
        self.left = None
        self.right = None


# BST class
class BST:
    def __init__(self):
        """
        Initialize empty BST
        """
        self.root = None

    def insert(self, root, data):
        """
        Recursively insert a node into BST

        Rules:
        1. If root is None, create new node
        2. If data < root.data, insert in left subtree
        3. If data > root.data, insert in right subtree
        """
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)
        else:
            root.right = self.insert(root.right, data)

        return root

    def inorder(self, root):
        """
        In-order traversal of BST

        Order:
        Left -> Root -> Right

        This prints elements in sorted order
        """
        if root:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)


# Main Program
bst = BST()

# Insert elements
bst.root = bst.insert(bst.root, 50)
bst.root = bst.insert(bst.root, 30)
bst.root = bst.insert(bst.root, 70)
bst.root = bst.insert(bst.root, 20)
bst.root = bst.insert(bst.root, 40)
bst.root = bst.insert(bst.root, 60)
bst.root = bst.insert(bst.root, 80)

# Display inorder traversal
print("In-order Traversal:")
bst.inorder(bst.root)
```

Output

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE

```
20 30 40 50 60 70 80
PS D:\AI coding\Lab-11> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/Lab-11/#Use AI to create a Binary Se
arch Tree (.py"
In-order Traversal:
20 30 40 50 60 70 80
PS D:\AI coding\Lab-11>
```

Python
Python
Python
Python

Observation

- BST was successfully implemented using recursive insert method.
- Nodes were inserted according to BST property.
- In-order traversal correctly displayed elements in sorted order.
- Recursive functions simplified insertion and traversal.
- BST improves search efficiency compared to linear structures.

Task Description #5 – Hash Table

Task: Use AI to implement a hash table with basic insert, search, and delete methods.

Sample Input Code:

class HashTable:
pass

Expected Output:

• Collision handling using chaining, with well-commented method

Prompt

Use AI to implement a Hash Table in Python with insert, search, and delete methods using collision handling through chaining.

#Use AI to create a Binary Search Tree (.py

#Use AI to create a Binary Search Tree (.py > ...

```python
#Use AI to implement a Hash Table in Python with insert, search, and delete methods using collision handling through chaining.
# Hash Table implementation using chaining

class HashTable:
    def __init__(self, size=10):
        """
        Initialize hash table with given size
        Each index contains a list (chain) to handle collisions
        """
        self.size = size
        self.table = [[] for _ in range(size)]

    def hash_function(self, key):
        """
        Generate index using modulo operation
        """
        return key % self.size

    def insert(self, key):
        """
        Insert key into hash table

        Steps:
        1. Find index using hash function
        2. Add key to chain if not already present
        """
        index = self.hash_function(key)

        if key not in self.table[index]:
            self.table[index].append(key)
            print(f"{key} inserted at index {index}")
        else:
            print(f"{key} already exists")

    def search(self, key):
        """
        Search key in hash table

        Returns True if found, else False
        """
        index = self.hash_function(key)

        if key in self.table[index]:
            print(f"{key} found at index {index}")
            return True
        else:
            print(f"{key} not found")
            return False

    def delete(self, key):
```

#Use AI to create a Binary Search Tree (.py

#Use AI to create a Binary Search Tree (.py > ...

```python
class HashTable:
            return False

    def delete(self, key):
        """
        Delete key from hash table
        """
        index = self.hash_function(key)

        if key in self.table[index]:
            self.table[index].remove(key)
            print(f"{key} deleted from index {index}")
        else:
            print(f"{key} not found for deletion")

    def display(self):
        """
        Display full hash table
        """
        print("\nHash Table:")
        for i in range(self.size):
            print(f"Index {i}: {self.table[i]}")


# Main Program
ht = HashTable(5)

# Insert elements
ht.insert(10)
ht.insert(15)
ht.insert(20)
ht.insert(25)
ht.insert(30)

# Display table
ht.display()

# Search elements
ht.search(20)
ht.search(99)

# Delete element
ht.delete(15)

# Display after deletion
ht.display()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS D:\AI coding\Lab-11> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/Lab-11/#Use AI to create a Binary Search Tree (.py"
10 inserted at index 0
15 inserted at index 0
20 inserted at index 0
25 inserted at index 0
30 inserted at index 0

Hash Table:
Index 0: [10, 15, 20, 25, 30]
Index 1: []
Index 2: []
Index 3: []
Index 4: []
20 found at index 0
99 not found
15 deleted from index 0

Hash Table:
Index 0: [10, 20, 25, 30]
Index 1: []
Index 2: []
Index 3: []
Index 4: []
PS D:\AI coding\Lab-11>
```

Observation

- Hash table was successfully implemented using chaining.
- Collision handling was achieved using lists at each index.
- Insert, search, and delete operations worked correctly.
- Hash function distributed elements across indices.
- Chaining allowed multiple elements at same index without data loss.
- Hash table provides faster access compared to linear search.

Task Description #6 – Graph Representation

Task: Use AI to implement a graph using an adjacency list.

Sample Input Code:

class Graph:

pass

Expected Output:

• Graph with methods to add vertices, add edges, and display connections.

Prompt

Use AI to implement a graph using an adjacency list in Python with methods to add vertices, add edges, and display graph connections.

#Use AI to create a Binary Search Tree (.py

#Use AI to create a Binary Search Tree (.py > ...

```python
#Use AI to implement a graph using an adjacency list in Python with methods to add vertices, add edges, and display graph connections.
# Graph implementation using adjacency list

class Graph:
    def __init__(self):
        """
        Initialize empty graph using dictionary
        key = vertex
        value = list of connected vertices
        """
        self.graph = {}

    def add_vertex(self, vertex):
        """
        Add a new vertex to the graph
        """
        if vertex not in self.graph:
            self.graph[vertex] = []
            print(f"Vertex {vertex} added")
        else:
            print(f"Vertex {vertex} already exists")

    def add_edge(self, v1, v2):
        """
        Add edge between v1 and v2 (undirected graph)
        """
        # Add vertices if not present
        if v1 not in self.graph:
            self.add_vertex(v1)
        if v2 not in self.graph:
            self.add_vertex(v2)

        # Add edge
        self.graph[v1].append(v2)
        self.graph[v2].append(v1)

        print(f"Edge added between {v1} and {v2}")

    def display(self):
        """
        Display adjacency list
        """
        print("\nGraph Adjacency List:")
        for vertex in self.graph:
            print(f"{vertex} -> {self.graph[vertex]}")


# Main Program
g = Graph()

# Add vertices
g.add_vertex(1)
g.add_vertex(2)
g.add_vertex(3)

# Add edges
g.add_edge(1, 2)
g.add_edge(1, 3)
g.add_edge(2, 3)

# Display graph
g.display()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

Index 4: []
PS D:\AI coding\Lab-11> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/Lab-11/#Use AI to create a Binary Search Tree (.py"
Vertex 1 added
Vertex 2 added
Vertex 3 added
Edge added between 1 and 2
Edge added between 1 and 3
Edge added between 2 and 3

Graph Adjacency List:
1 -> [2, 3]
2 -> [1, 3]
3 -> [1, 2]
PS D:\AI coding\Lab-11>
```

Observation

- Graph was successfully implemented using adjacency list.
- Vertices and edges were added correctly.
- Adjacency list stored connections efficiently.
- Display method correctly showed graph structure.
- Adjacency list uses less memory compared to adjacency matrix.
- Graph representation supports dynamic addition of vertices and edges.

Task Description #7 – Priority Queue

Task: Use AI to implement a priority queue using Python's heapq module.

Sample Input Code:

class PriorityQueue:

pass

Expected Output:

• Implementation with enqueue (priority), dequeue (highest priority), and display methods.

Prompt

Use AI to implement a Priority Queue in Python using the heapq module with enqueue, dequeue, and display methods.

#Use AI to create a Binary Search Tree (.py

#Use AI to create a Binary Search Tree (.py > ...

```python
#Use AI to implement a Priority Queue in Python using the heapq module with enqueue, dequeue, and display methods.
import heapq

class PriorityQueue:
    def __init__(self):
        """
        Initialize empty priority queue using heap
        """
        self.heap = []

    def enqueue(self, element, priority):
        """
        Insert element with given priority

        Lower number = Higher priority
        heapq maintains min heap property
        """
        heapq.heappush(self.heap, (priority, element))
        print(f"Inserted {element} with priority {priority}")

    def dequeue(self):
        """
        Remove and return element with highest priority
        """
        if not self.heap:
            print("Priority Queue is empty")
            return None

        priority, element = heapq.heappop(self.heap)
        print(f"Removed {element} with priority {priority}")
        return element

    def display(self):
        """
        Display all elements in priority queue
        """
        print("\nPriority Queue Elements:")
        for priority, element in self.heap:
            print(f"Element: {element}, Priority: {priority}")


# Main Program
pq = PriorityQueue()

# Insert elements
pq.enqueue("Task A", 3)
pq.enqueue("Task B", 1)
pq.enqueue("Task C", 2)
pq.enqueue("Task D", 4)

# Display queue
pq.display()

# Remove highest priority element
pq.dequeue()

# Display after removal
pq.display()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

3 -> [1, 2]
PS D:\AI coding\Lab-11> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/Lab-11/#Use AI to create a Binary Search Tree (.py"
Inserted Task A with priority 3
Inserted Task B with priority 1
Inserted Task C with priority 2
Inserted Task D with priority 4

Priority Queue Elements:
Element: Task B, Priority: 1
Element: Task A, Priority: 3
Element: Task C, Priority: 2
Element: Task D, Priority: 4
Removed Task B with priority 1

Priority Queue Elements:
Element: Task C, Priority: 2
Element: Task A, Priority: 3
Element: Task D, Priority: 4
PS D:\AI coding\Lab-11>
```

Observation

- Priority Queue was successfully implemented using heapq module.
- Elements were inserted based on priority.
- Highest priority element was removed first.
- heapq maintained heap property automatically.
- Priority Queue is useful in scheduling and task management.

Task Description #8 – Deque

Task: Use AI to implement a double-ended queue using collections.deque.

Sample Input Code:

class DequeDS:

pass

Expected Output:

• Insert and remove from both ends with docstrings.

Prompt

Use AI to implement a double-ended queue using Python’s collections.deque with methods to insert and remove elements from both front and rear.

#Use AI to create a Binary Search Tree (.py

D:\AI coding\Lab-11\#Use AI to create a Binary Search Tree (.py

```python
#Use AI to implement a double-ended queue using Python’s collections.deque with methods to insert and remove elements from both
from collections import deque

class DequeDS:
    def __init__(self):
        """
        Initialize empty deque
        deque allows insertion and deletion from both ends
        """
        self.dq = deque()

    def insert_front(self, element):
        """
        Insert element at the front of deque
        """
        self.dq.appendleft(element)
        print(f"{element} inserted at front")

    def insert_rear(self, element):
        """
        Insert element at the rear of deque
        """
        self.dq.append(element)
        print(f"{element} inserted at rear")

    def remove_front(self):
        """
        Remove element from front of deque
        """
        if not self.dq:
            print("Deque is empty")
            return None

```

#Use AI to create a Binary Search Tree (.py

#Use AI to create a Binary Search Tree (.py > ...

```python
class DequeDS:
    def remove_front(self):
        """
        if not self.dq:
            print("Deque is empty")
            return None

        element = self.dq.popleft()
        print(f"{element} removed from front")
        return element

    def remove_rear(self):
        """
        Remove element from rear of deque
        """
        if not self.dq:
            print("Deque is empty")
            return None

        element = self.dq.pop()
        print(f"{element} removed from rear")
        return element

    def display(self):
        """
        Display all elements in deque
        """
        print("\nDeque elements:")
        print(list(self.dq))


# Main Program
dq = DequeDS()

# Insert elements
dq.insert_front(10)
dq.insert_rear(20)
dq.insert_front(5)
dq.insert_rear(30)

# Display deque
dq.display()

# Remove elements
dq.remove_front()
dq.remove_rear()

# Display after removal
dq.display()
```

Output

```
> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/Lab-11/#Use AI to create a Binary Search Tree (.py"
10 inserted at front
20 inserted at rear
5 inserted at front
30 inserted at rear

Deque elements:
[5, 10, 20, 30]
5 removed from front
30 removed from rear

Deque elements:
[10, 20]
PS D:\AI coding\Lab-11>
```

Observation

- Deque was successfully implemented using collections.deque.
- Elements were inserted and removed from both ends.
- appendleft() and append() handled insertion efficiently.
- popleft() and pop() handled deletion efficiently.
- Deque provides faster operations than list for both ends.

Task Description #9

Real-Time Application Challenge – Choose theRight Data Structure

Scenario:

Your college wants to develop a Campus Resource Management System that handles:

1. Student Attendance Tracking – Daily log of students entering/exiting the campus.
2. Event Registration System – Manage participants in events with quick search and removal.
3. Library Book Borrowing – Keep track of available books and their due dates.
4. Bus Scheduling System – Maintain bus routes and stop connections.
5. Cafeteria Order Queue – Serve students in the order they arrive.

Student Task:

• For each feature, select the most appropriate data structure from the list below:

o Stack

o Queue
o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque

• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen featurewith comments and docstrings.

Prompt

Select appropriate data structures for campus management features, justify your choices, and implement one feature using Python.

#Use AI to create a Binary Search Tree (.py

```python
from collections import deque

class CafeteriaQueue:
    """
    Queue implementation for cafeteria order management
    FIFO order is maintained
    """

    def __init__(self):
        """Initialize empty queue"""
        self.queue = deque()

    def place_order(self, student_name):
        """
        Add student order to queue
        """
        self.queue.append(student_name)
        print(f"Order placed by {student_name}")

    def serve_order(self):
        """
        Serve student order from front of queue
        """
        if not self.queue:
            print("No orders to serve")
            return None

        student = self.queue.popleft()
        print(f"Order served to {student}")
        return student

    def display_orders(self):
        """
        Display all pending orders
        """
        print("\nCurrent Orders in Queue:")
        for student in self.queue:
            print(student)


# Main Program
cq = CafeteriaQueue()

# Place orders
cq.place_order("Anish")
cq.place_order("Rahul")
cq.place_order("Sneha")
cq.place_order("Priya")

# Display orders
cq.display_orders()

# Serve orders
cq.serve_order()
cq.serve_order()

# Display remaining orders
cq.display_orders()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

PS D:\AI coding\Lab-11> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/Lab-11/#Use AI to create a Binary Search Tree (.py"
Order placed by Anish
Order placed by Rahul
Order placed by Sneha
Order placed by Priya

Current Orders in Queue:
Anish
Rahul
Sneha
Priya
Order served to Anish
Order served to Rahul

Current Orders in Queue:
Sneha
Priya
PS D:\AI coding\Lab-11>
```

Observation

- Queue data structure was successfully used to manage cafeteria orders.
- Orders were processed in FIFO order.
- New orders were added at rear and served from front.
- Queue ensured fair and efficient order handling.
- collections.deque provided efficient queue operations.

Task Description #10:

Smart E-Commerce Platform – Data StructureChallenge

An e-commerce company wants to build a Smart Online Shopping System with:
1. Shopping Cart Management – Add and remove products dynamically.
2. Order Processing System – Orders processed in the order they are placed.
3. Top-Selling Products Tracker – Products ranked by sales count.
4. Product Search Engine – Fast lookup of products using product ID.
5. Delivery Route Planning – Connect warehouses and delivery locations.

Student Task:
• For each feature, select the most appropriate data structure from the list below:
o Stack
o Queue

o Priority Queue
o Linked List
o Binary Search Tree (BST)
o Graph
o Hash Table
o Deque
• Justify your choice in 2–3 sentences per feature.
• Implement one selected feature as a working Python program with AI-assisted code generation.

Expected Output:

• A table mapping feature → chosen data structure → justification.
• A functional Python program implementing the chosen feature with comments and docstrings.

Prompt

Select appropriate data structures for e-commerce features, justify your selection, and implement one feature using Python.

#Use AI to create a Binary Search Tree (.py

```python
from collections import deque

class OrderQueue:
    """
    Queue implementation for order processing system
    Orders are processed in FIFO order
    """

    def __init__(self):
        """Initialize empty order queue"""
        self.orders = deque()

    def place_order(self, order_id):
        """
        Add new order to queue
        """
        self.orders.append(order_id)
        print(f"Order placed: {order_id}")

    def process_order(self):
        """
        Process order from front of queue
        """
        if not self.orders:
            print("No orders to process")
            return None

        order = self.orders.popleft()
        print(f"Order processed: {order}")
        return order

    def display_orders(self):
        """
        Display all pending orders
        """
        print("\nPending Orders:")
        for order in self.orders:
            print(order)


# Main Program
oq = OrderQueue()

# Place orders
oq.place_order("ORD101")
oq.place_order("ORD102")
oq.place_order("ORD103")

# Display orders
oq.display_orders()

# Process orders
oq.process_order()
oq.process_order()

# Display remaining orders
oq.display_orders()
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    POSTMAN CONSOLE

ORD103
PS D:\AI coding\Lab-11> & C:/Users/ravul/AppData/Local/Programs/Python/Python312/python.exe "d:/AI coding/Lab-11/#Use AI to create a Binary Search Tree (.py"
Order placed: ORD101
Order placed: ORD102
Order placed: ORD103

Pending Orders:
ORD101
ORD102
ORD103
Order processed: ORD101
Order processed: ORD102

Pending Orders:
ORD103
PS D:\AI coding\Lab-11>
```

Observation

- Queue data structure was successfully used for order processing.
- Orders were processed in FIFO order.
- New orders were added at rear and processed from front.
- Queue ensured correct and fair order handling.
- deque provided efficient queue operations.